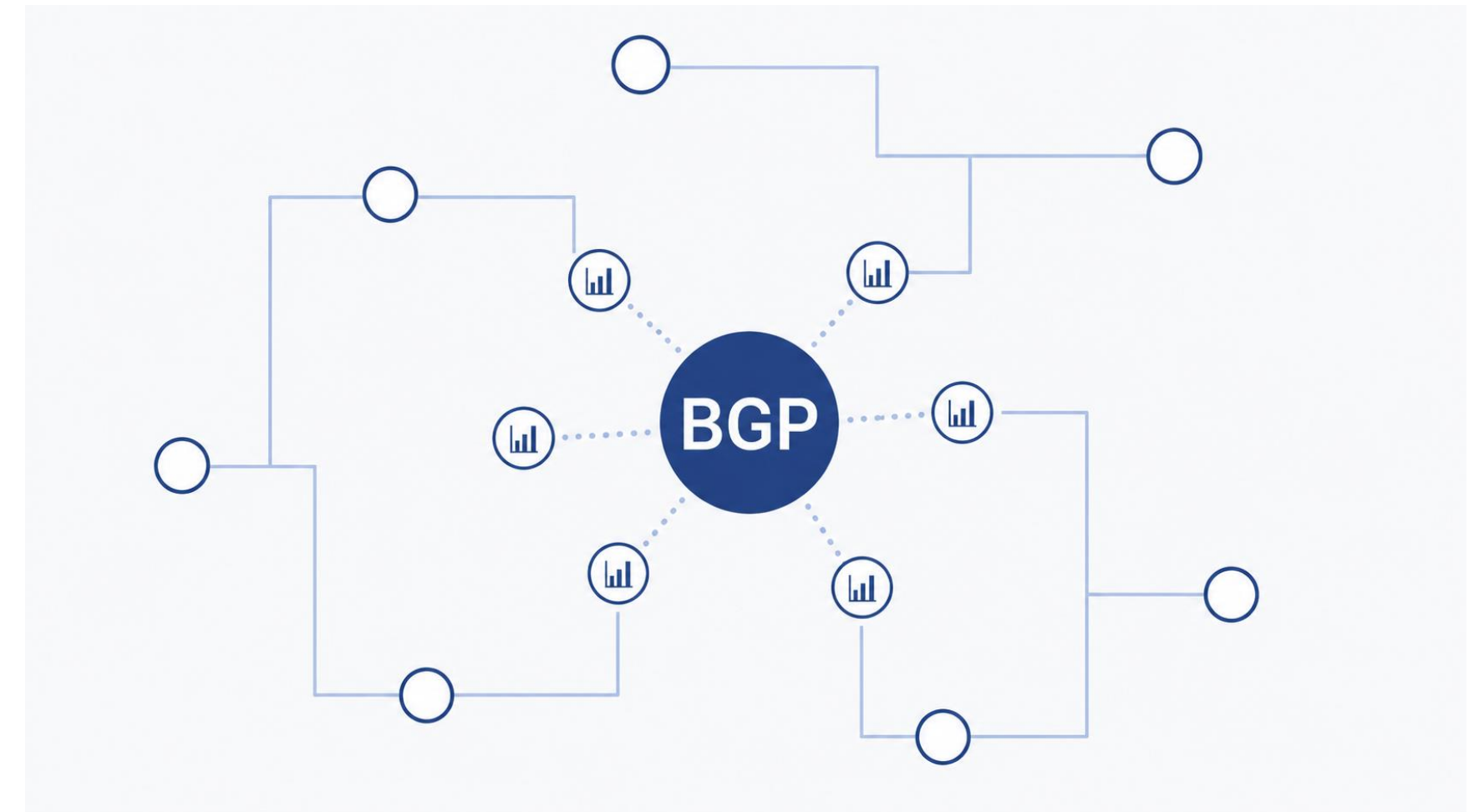


BGP Control Plane Measurement

Week 2: from the global routing table to **who owns** and **who reaches** the Internet's address space.



PLENARY

Tue Jun 23 · 12:00 PT

INSTRUCTOR

kc claffy

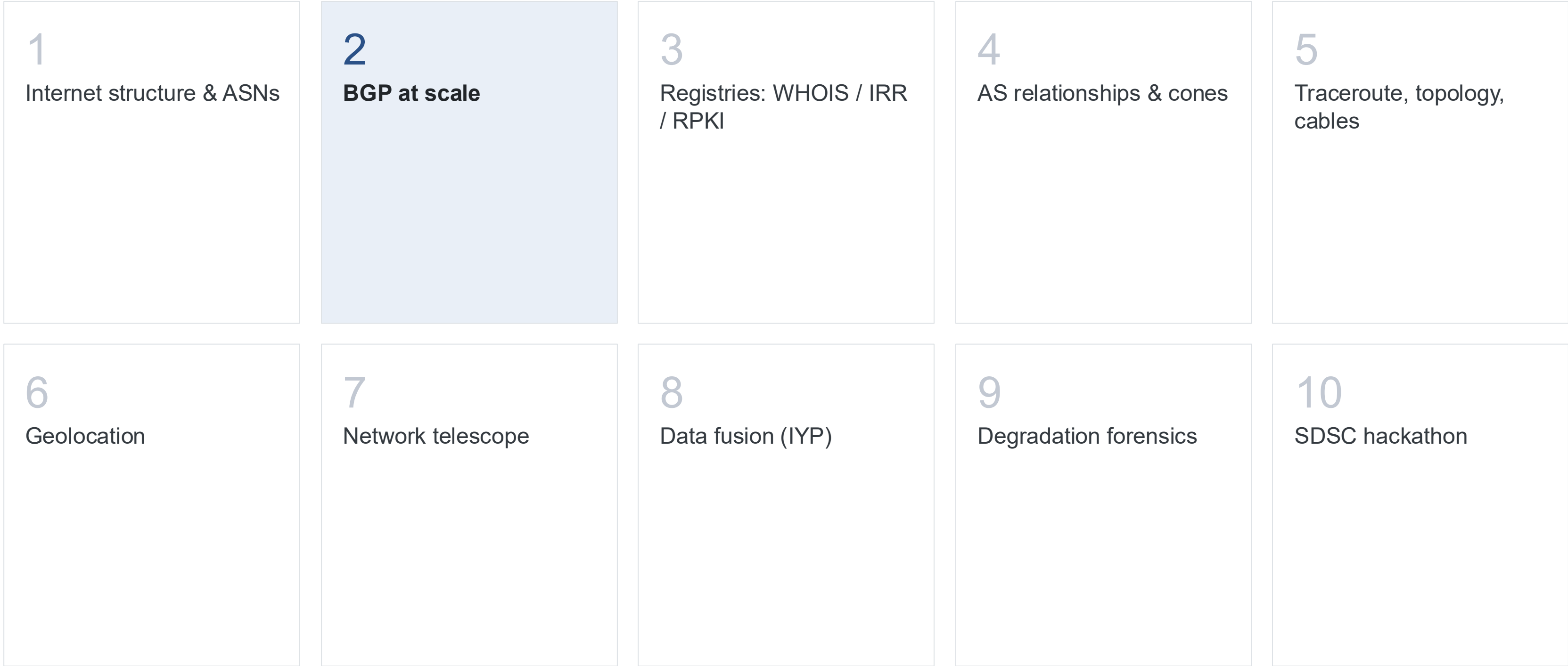
SENIOR MENTORS

Ricky Mok · Bradley Huffaker · Rocky Chang

PROGRAM CONTACT

Alex Ma

Ten building blocks — you are here



Weeks 1–8 build one block at a time; weeks 9–10 integrate them into a full forensic pipeline.
ESCALATE Training Program · Summer 2026

From topology to address space

- Block 1 mapped Internet structure — ASes, ASNs, and how they interconnect.
- You worked the topology view: who connects to whom.
- We deferred the next question: who owns and who reaches the address space.

THIS WEEK BUILDS ON IT

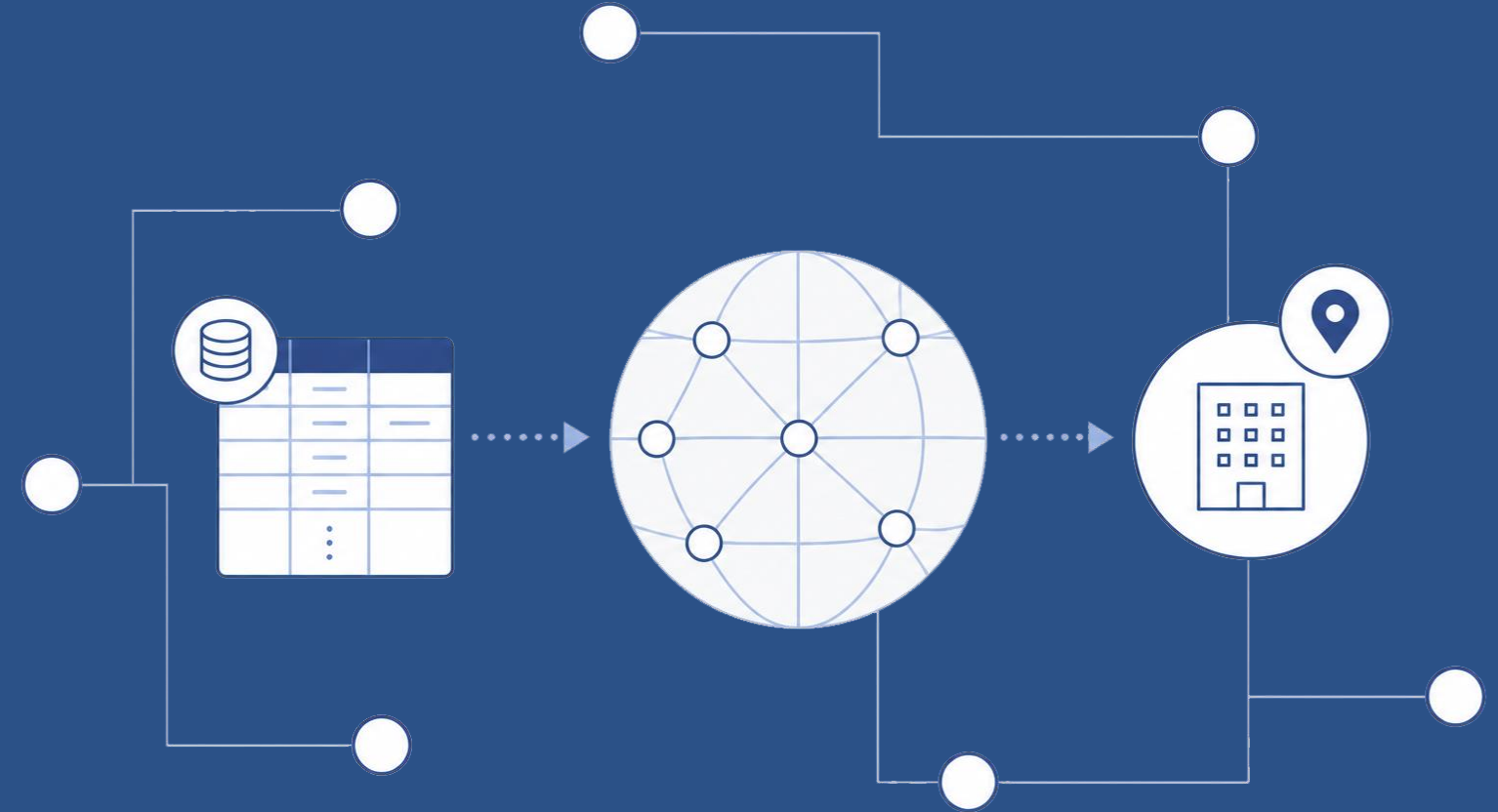
We read the global routing table to attribute every prefix to an origin AS — then to the cone that reaches it.

PART ONE

01

From RIB to origin AS

Reading the global routing table to attribute address space.



BGP as a system, not just a protocol

- BGP is a **protocol** for inter-domain routing — we analyze it as a **system** of global reachability.
- Measurement: how do we see the global routing state at one moment?
- RIB (Routing Information Base): a collector's snapshot of every route it has received.



Datasets & tools for this lab

RouteViews / MRT Global routing tables archived by Univ. of Oregon collectors, as MRT (Multi-Threaded Routing Toolkit) binary records.	COLLECTOR · RIB <code>route-views3 · 2026.05</code>
bgpkit Decodes MRT so you iterate over routing records as Python objects.	TOOL <code>pybgpkit_parser</code>
CAIDA customer cones (PPDC) Per-AS customer-cone membership, derived from provider→customer links.	FILE <code>20260501.ppdcc-ases.txt.bz2</code>
AS2Org Maps ASNs to real-world organizations and country.	STORED AS <code>as2org.jsonl</code>

Anatomy of a RIB entry

Each MRT record is one prefix announcement seen from one vantage point (**peer**):

Field	Example	Meaning
collector	route-views3	which collector
peer_asn / peer_ip	10 / 12.3.2.1	the vantage point
as_path	10 724 8234	ASNs the route traversed
origin_asns	8234	last ASN — the originator
prefix	192.0.2.0/24	the announced CIDR block

The **origin AS** — the last ASN — is what we attribute address space to.

The AS path: origin and MOAS

- Take the **last ASN** in the path — the origin AS, via `elem.origin_asns`.
- Attribution: each prefix is credited to its origin AS — this drives the Task 1 counts.
- MOAS: the same prefix shows **different origins** across the table.

```
# as_path: 10 724 8234 -> origin = last hop
for elem in bgpkit.Parser(url=url):
    prefix_origins[elem.prefix] \
        .update(elem.origin_asns)
```

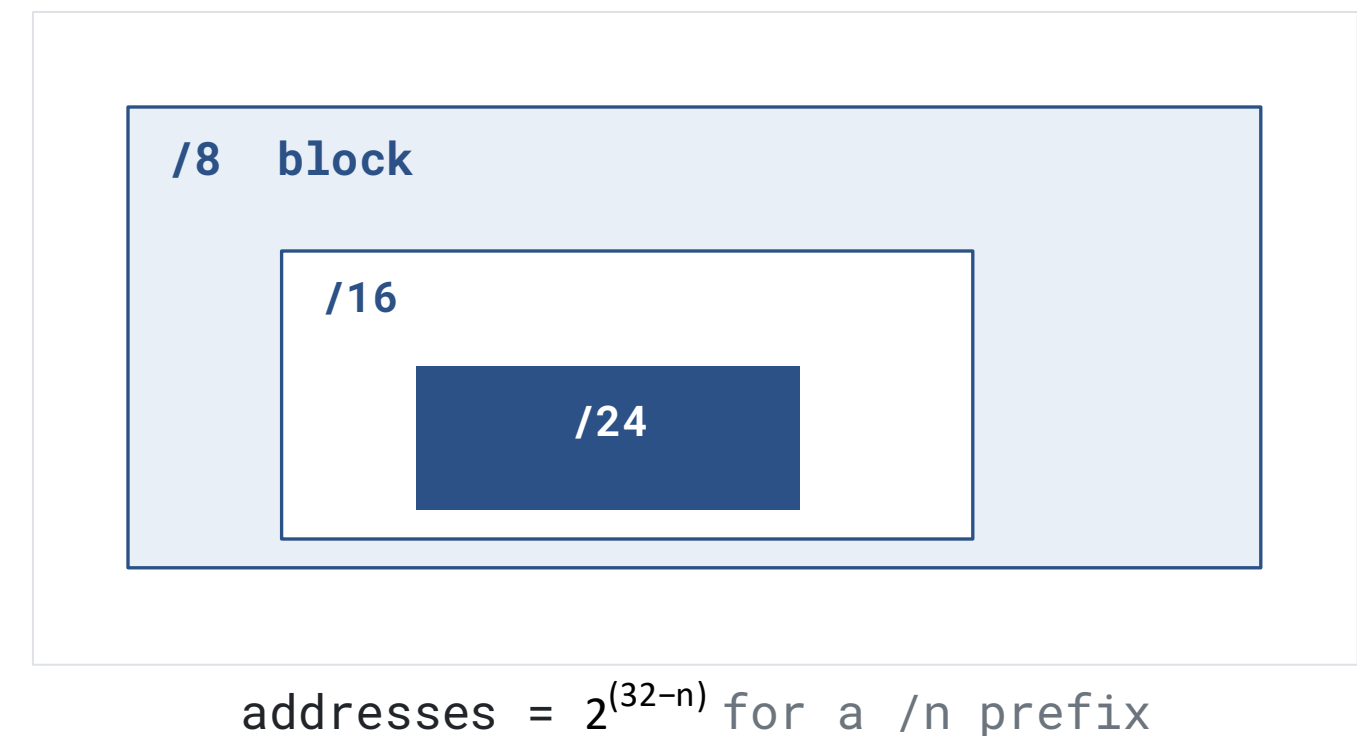

The Multi-Origin AS puzzle

- A **MOAS** prefix is announced from more than one AS at once.
- Benign: multi-homing, traffic engineering, mergers / acquisitions.
- **Security risk:** can signal a BGP hijack — an unauthorized AS announcing someone else's space.
- Task rule: exclude MOAS from per-AS counting (can't attribute cleanly). Ballpark < 1%.

```
if len(origins) > 1:
    moas_count += 1
else:
    asn = next(iter(origins))
```

Prefixes vs. address space

- Counting *prefixes* is coarse — prefix size varies exponentially.
- $/8 \rightarrow 16,777,216$ addresses $/16 \rightarrow 65,536$ $/24 \rightarrow 256$
- A few coarse blocks can dwarf thousands of $/24$ s — we need an **address count**.

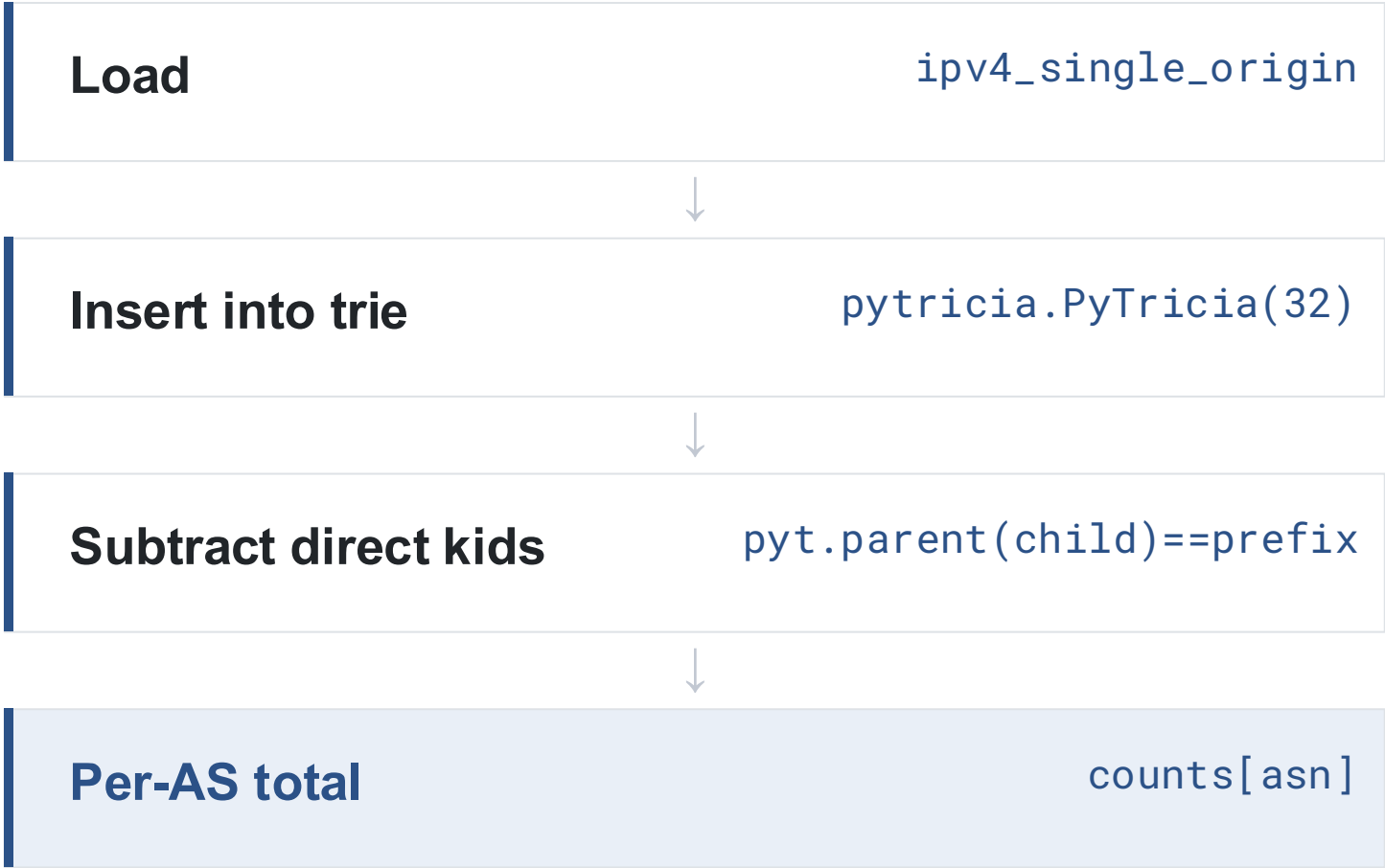


Counting addresses: longest-prefix match

Prefixes nest. Credit each address to the **most-specific** prefix covering it: take a prefix's size and subtract the sizes of its direct children only.

THE HARDEST CORRECTNESS POINT

Direct children only. If $A \supset B \supset C$, A subtracts B (which already contains C); B subtracts C. Subtracting C from A too undercounts A.



Longest-prefix match with PyTricia

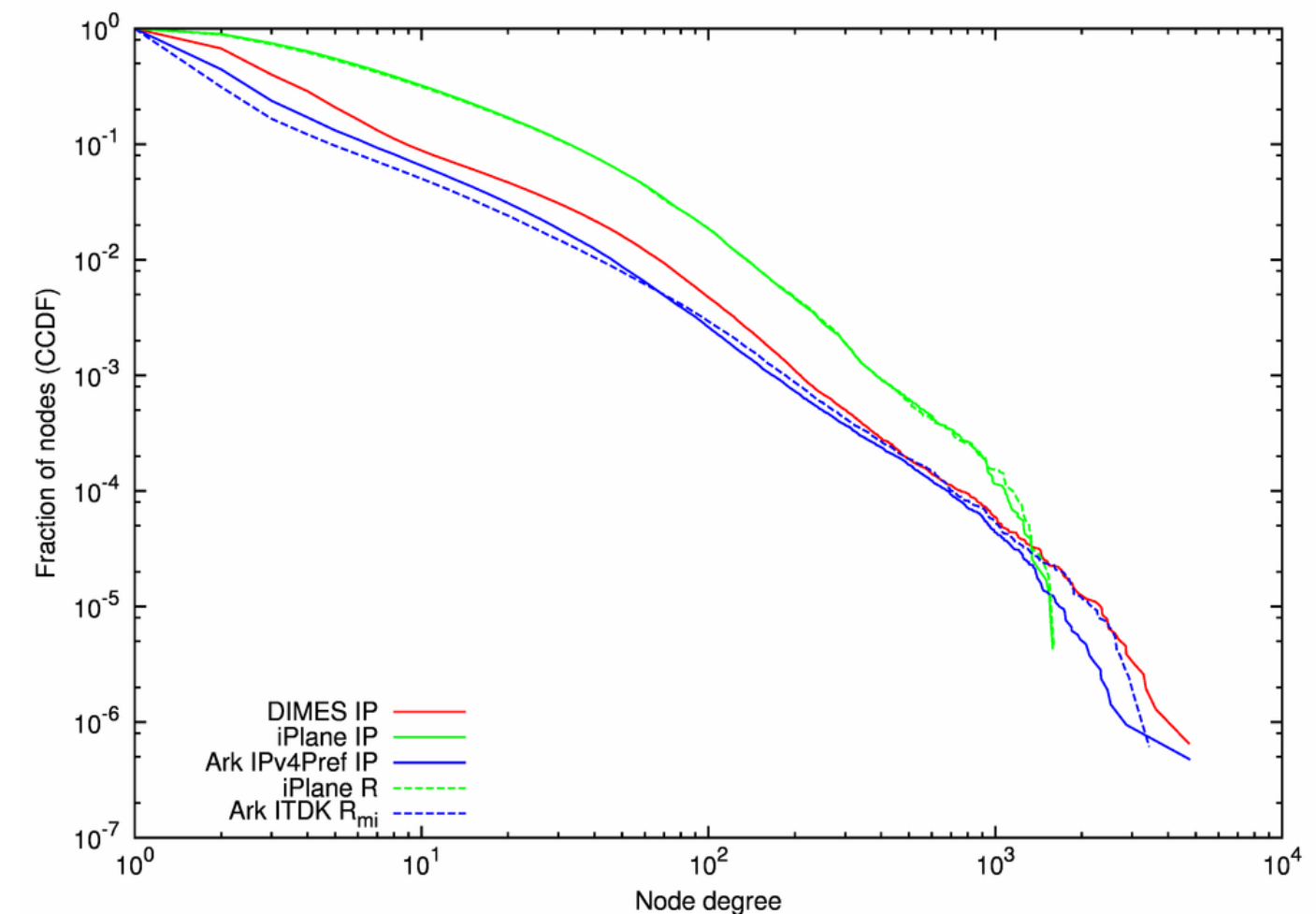
```
pyt = pytricia.PyTricia(32)
for prefix, asn in ipv4_single_origin:
    pyt[prefix] = asn

for prefix, asn, plen in parsed:
    total = 1 << (32 - plen)
    overlap = 0
    for child in pyt.children(prefix):
        if child == prefix: continue          # children() includes self
        if pyt.parent(child) == prefix:      # DIRECT children only
            overlap += 1 << (32 - int(child.split('/')[1]))
    counts[asn] += total - overlap
```

Scale check: total attributed IPv4 $\approx$ 3.1 billion across $\approx$ 77,600 origin ASes (not $\approx$ 4.3B — that means double-counting).

Reading a CCDF

- A **CCDF** plots, for each value x , the number of ASes with $\geq x$.
- Log-log axes — counts span 8+ orders of magnitude.
- Steep left + long tail = a few heavy hitters dominate; a near-straight line $\approx$ power law.



CCDF of Node Degree

Origin metrics, and Q1 / Q3

Metric	min	max	mean	median
Origin prefix count	1	~14.4k	~14	2
Origin address count	8	~223M	—	—

Q1 — shape: heavy-tailed; most ASes tiny (median 2), a few enormous (mean 14, dragged by the tail).

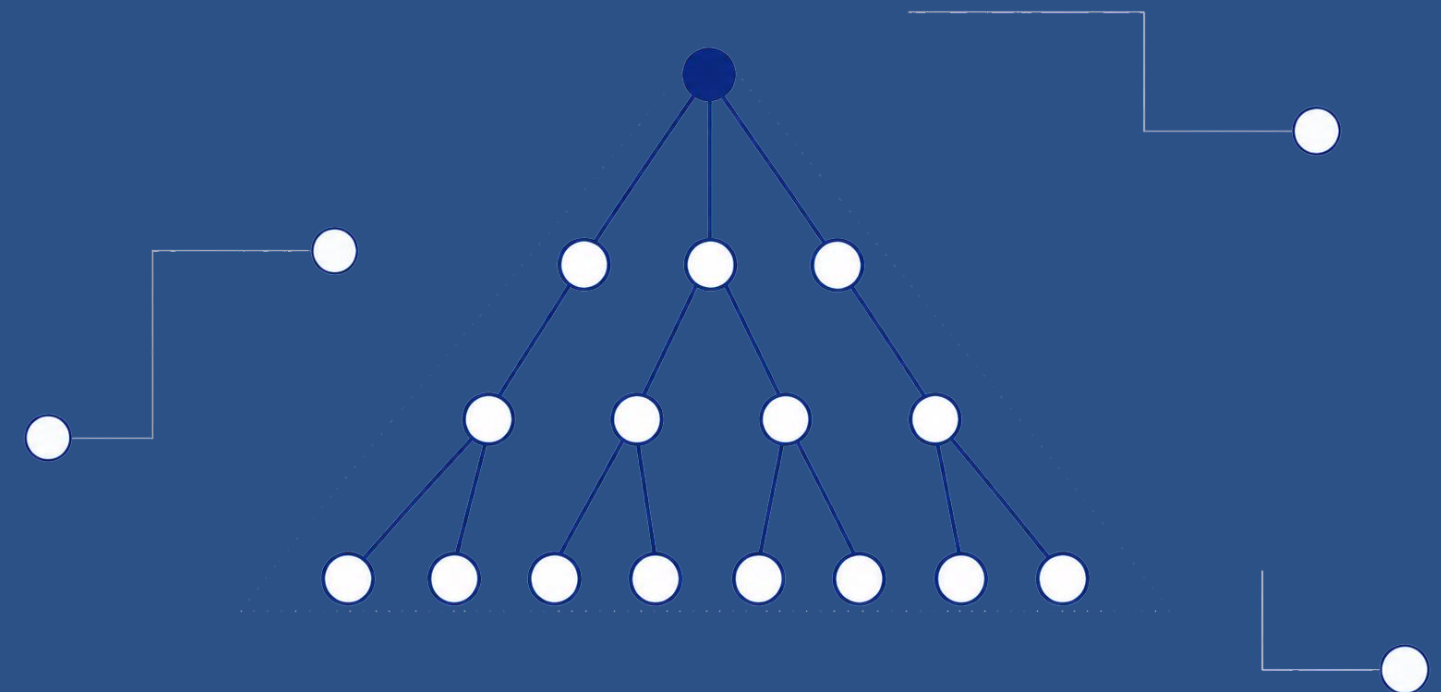
Q3 — divergence: the curves separate because prefix size varies hugely — **the top-prefix AS is not the top-address AS.**

PART TWO

02

Customer cones & reach

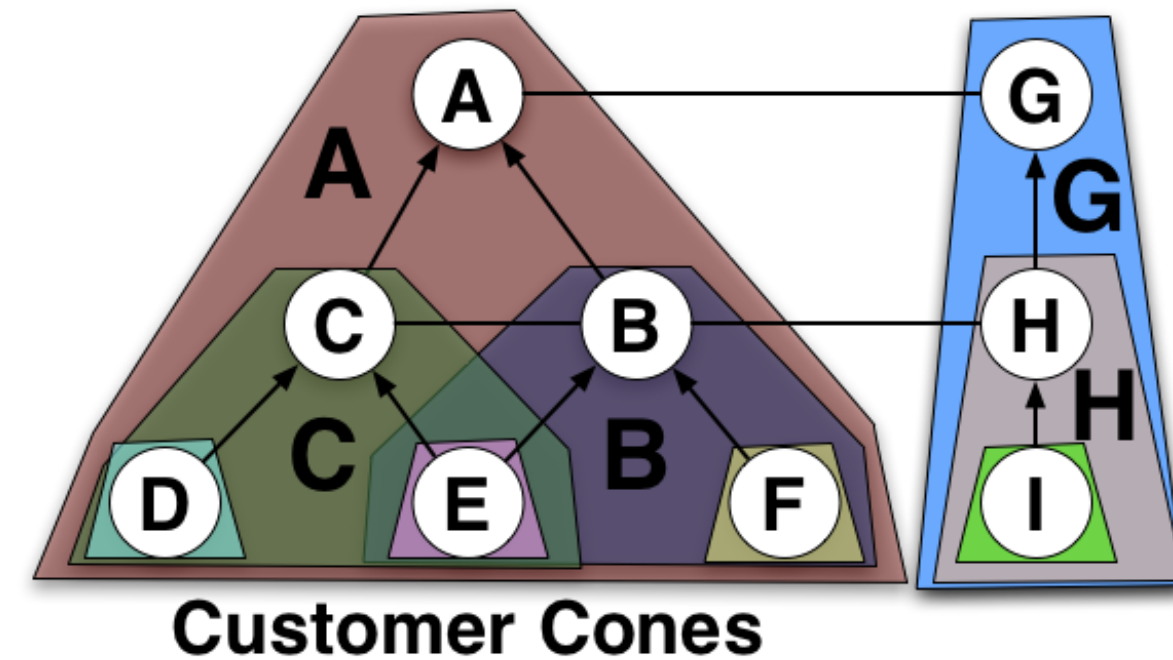
From what an AS announces to what its cone can reach.



The customer cone

- Customer cone = the root AS plus every AS reachable via **provider**→**customer** links.
- Measures **transit influence** — not just what an AS announces itself.
- Input: CAIDA PPDC file `20260501.ppdcc-ases.txt.bz2`, loaded as a black box.

You **use** the precomputed cone file here; **how** relationships & cones are inferred is Block 4.



This graph shows the respective cone sizes for each AS

Owens vs. reaches

Task 1 — origin

what an AS owns

- **Question:** who owns the space?
- **Nested more-specifics:** subtracted (LPM)
- **Method:** PyTricia children-subtraction

Task 2 — cone

what an AS reaches

- **Question:** what does the cone reach?
- **Nested more-specifics:** **NOT subtracted**
- **Method:** sorted-interval sweep

Counting the cone: prefix sum + address sweep

Cone prefix count — sum members' prefix counts

```
cone_prefix_count[root] = sum(
    asn_prefix_count.get(m, 0)
    for m in members)
```

Cone address count — merge nets into runs

```
nets = sorted(member_nets, key=...)
total, covered_end = 0, -1
for net in nets:
    start = int(net.network_address)
    if start > covered_end:
        total += net.num_addresses
        covered_end = start + \
            net.num_addresses - 1
```

Within a cone we ask “is this reachable through me?” — so we do not subtract more-specifics. The deliberate opposite of Task 1.

Cone metrics, and Q4 / Q5

Metric	min	max	mean	median
Cone prefix count	1	~893k	~203	3
Cone address count	8	~2.35B	—	~1k

Q4 — shape: even more heavy-tailed (median cone 3; the top cone reaches hundreds of thousands).

Q5 — why cone ≥ origin: the cone is a superset of the AS itself (equal only for stubs). Providers reach customer prefixes they don't originate — so cone count dwarfs origin count.

PART THREE

03

From numbers to organizations

Who are the leaders — and why they rank where they do.

Map ASNs to organizations (AS2Org)

- Join the numeric leaders back to real-world orgs using CAIDA `as2org.jsonl` (orgName, country, members).
- One organization can hold many ASNs — clustered from registry records.

Kind	Example	Ranks high via
Transit backbone	Lumen / L3	huge cone
Cloud / content	Amazon	own prefixes / addresses
Legacy holder	US DoD	coarse address blocks

The rank-vector sort

1. Rank each metric independently (rank 1 = highest).
2. Union the top-3 ASNs across the four rank dicts.
3. Sort rows by each ASN's **sorted vector of ranks** — best first.

```
def ranked(d):  
    return {asn: r for r, (asn, _) in  
            enumerate(sorted(d.items(),  
                             key=lambda x: x[1],  
                             reverse=True), start=1)}
```

[1,1,3,4] beats **[1,2,2,3]** beats **[2,2,3,9999]** — compare best rank, then next. Being #1 in two metrics beats #1 in one.

Q6: one sentence per top-4 organization

- **Transit backbone** — ranks via cone (prefixes + addresses), needn't own much itself.
- **Cloud / content** — ranks via own address count (large hosting allocations).
- **Legacy holder** — ranks via address count from historic coarse blocks, modest cone.

*The exact four orgs and counts are **[student computes]** — name the mechanism, not just the org.*

Common pitfalls

- **Don't double-count nested space** — total $\approx 3.1\text{B}$, not $\approx 4.3\text{B}$.
- **Task 2 sweep does NOT subtract more-specifics** — the opposite of Task 1.
- Use `.update(origin_asns)` not `.add()`; exclude MOAS; CCDF $y = \text{ASes with } \geq x$ (log–log).
- Ranking unions top-3 across metrics, sorted by the rank vector.

nids-bgp-control-plane

Attribute address space from the global routing table, measure cone reach, and rank the leaders — the BGP section of your module.

- Task 1 — origin map, MOAS %, LPM address counts, origin CCDF (Q1–Q3)
- Task 2 — customer-cone prefix & address counts (Q4–Q5)
- Task 3 — top-3-in-any-metric table joined to AS2Org (Q6)
- Finish all notebook tasks and questions

MODULE MILESTONE

Becomes the BGP / control-plane section of your own training module.

REPOSITORY

`github.com/CAIDA/
nids-bgp-control-plane`

This week, in five steps

- Open the notebook on NRP JupyterHub (jupyterhub-west.nrp-nautilus.io).
- Work Tasks 1–3; replace each `# YOUR CODE HERE`.
- Finish the assignment by the beginning of Week 3.
- Attend the plenary — or catch up; materials are posted.
- Use the hands-on hour and mentors for infrastructure help.

OPTIONAL BACKGROUND

RouteViews & MRT background;
CAIDA AS Rank notes on customer
cones — context, returns in Block 4.

Questions · Mattermost (fastest)
training-program-info@caida.org

APPENDIX · FOR SELF-STUDY

A

Supporting detail, not covered live

A few slides so the deck stands on its own.

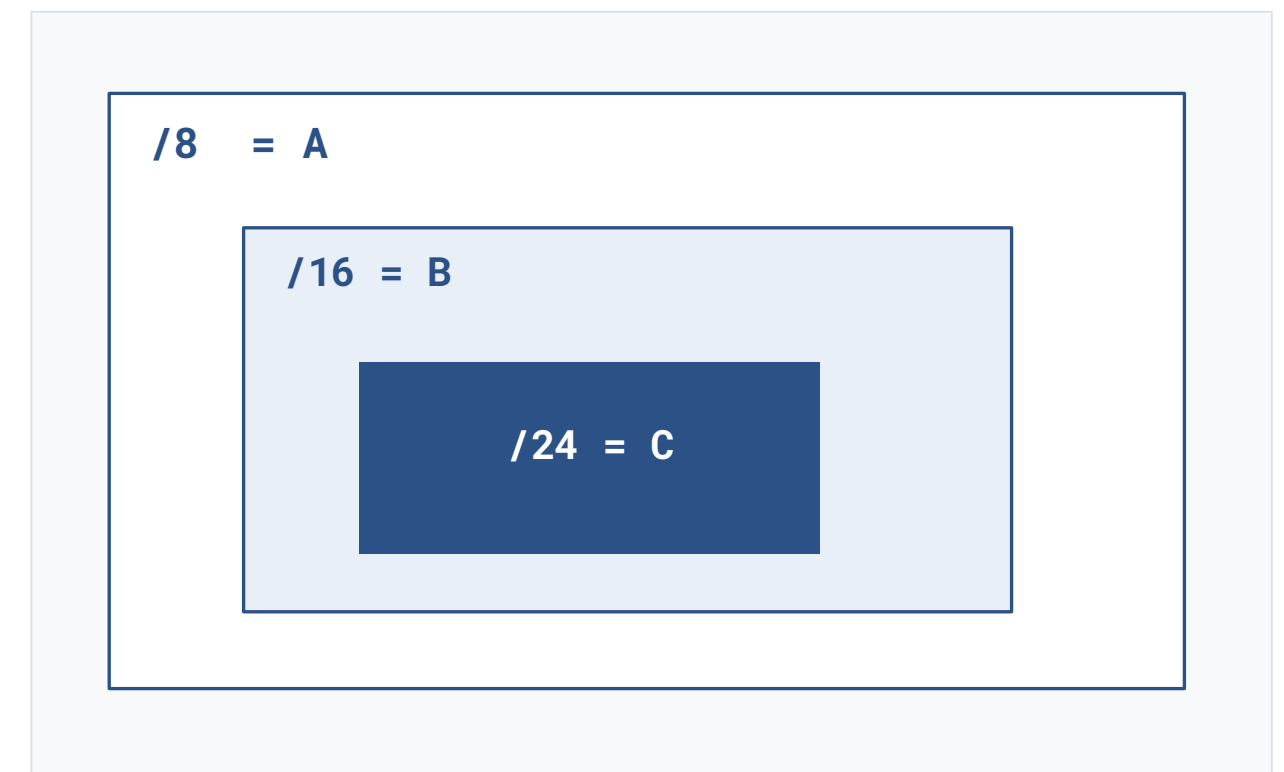
Longest-prefix match, three levels deep

Prefixes nest arbitrarily deep. Each address must be credited once, to the most-specific prefix that covers it, so a provider's block is not counted on top of the customer more-specifics inside it.

Subtract direct children only. With $A \supset B \supset C$: A subtracts B (B already contains C), and B separately subtracts C. If A also subtracted C, A would be undercounted — C would be removed twice.

A = 10.0.0.0/8 (16,777,216)
B = 10.1.0.0/16 (65,536, in A)

A = 16,777,216 - 65,536 = 16,711,680
B = 65,536



Advanced terms and acronyms

Program-, platform-, and research-specific terms used in this deck. (General terms like IPv4, BGP, AS, CIDR omitted.)

AS2Org

CAIDA dataset mapping ASNs to organizations (orgName, country, members).

bgpkit

Toolkit that parses MRT routing dumps into Python objects.

CCDF

Complementary CDF: for each x , the count of items with value $\geq x$.

customer cone

Root AS plus all ASes reachable via provider→customer links.

LPM

Longest-prefix match: credit an address to its most-specific covering prefix.

MOAS

Multi-Origin AS: one prefix announced by more than one origin AS.

MRT

Multi-Threaded Routing Toolkit — binary format for archived routing tables.

origin AS

The last ASN in a BGP path — who announced the prefix.

OSDF

Open Science Data Federation — staging path used to fetch the RIB.

PPDC

Per-AS customer-cone membership file (provider→customer derived).

RIB

Routing Information Base — a collector's snapshot of received routes.

RouteViews

Univ. of Oregon project archiving BGP tables from many collectors.